To Train Them Is My Cause: Exploring Pokémon Battles

Group Info

Jothiswaran Arumugam (JA23L)
Likhith Kumar Yaramala (LY23B)
Nandhini Muralidharan (NM23H)
Niveda Giridharan (NG23F)

Introduction

The world of Pokémon has captivated millions of fans worldwide, with its diverse array of unique creatures, each with distinct characteristics and capabilities. Understanding the factors that contribute to a Pokémon's overall strength and performance is of great interest to both players and researchers alike.

Motivation

Winning percentage is a critical metric that reflects a Pokémon's overall combat powers and viability within the game. Identifying the attributes and strategies that lead to higher winning percentages can inform team building and battle tactics, allowing players to optimize their chances of success. From the game developer's perspective, uncovering the underlying mechanics that influence winning percentage can inform balancing efforts, ensuring the game remains engaging and competitive while maintaining a sense of fairness and progression. By investigating the predictors of winning percentage, this study aims to provide valuable insights that can be leveraged to enhance the player experience, foster strategic depth, and contribute to the long-term sustainability and growth of the Pokémon gaming ecosystem.

Hypotheses

Based on the existing literature and the characteristics of the Pokemon dataset, we propose the following hypotheses:

1. The individual stats of a Pokemon, such as HP, Attack, and Defense, will be significant predictors of its winning percentage in battles.
2. The type (primary and secondary) of a Pokemon will have a significant influence on its winning percentage, with certain type combinations exhibiting higher or lower winning percentages.
3. Legendary Pokemon will have significantly higher winning percentages compared to non-legendary Pokemon, even after accounting for other predictor variables.
4. There will be nonlinear relationships or interaction effects between the predictor variables (stats, types, etc.) that contribute to the explanation of winning percentage.
5. The importance and influence of the predictor variables on winning percentage will vary across different battle formats or gameplay scenarios (e.g., single battles, double battles, online vs. offline).

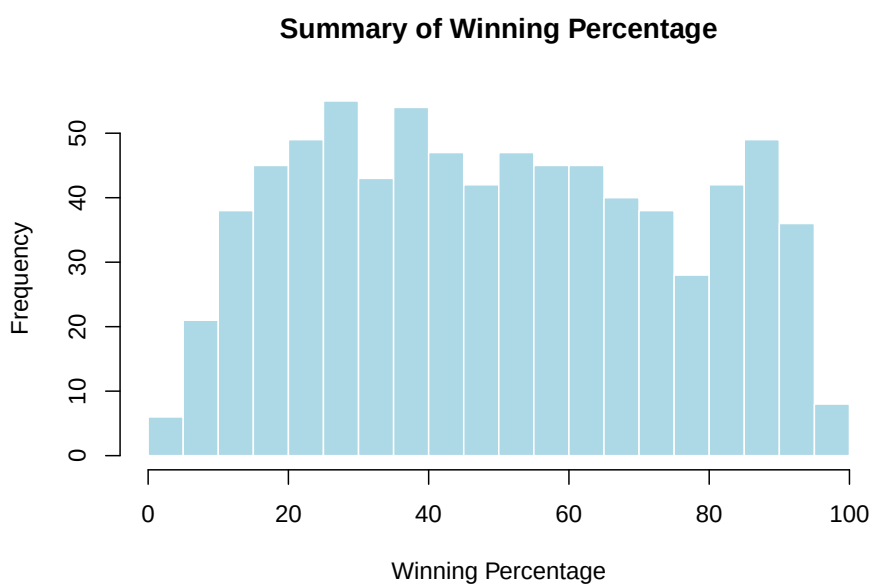
Data Description

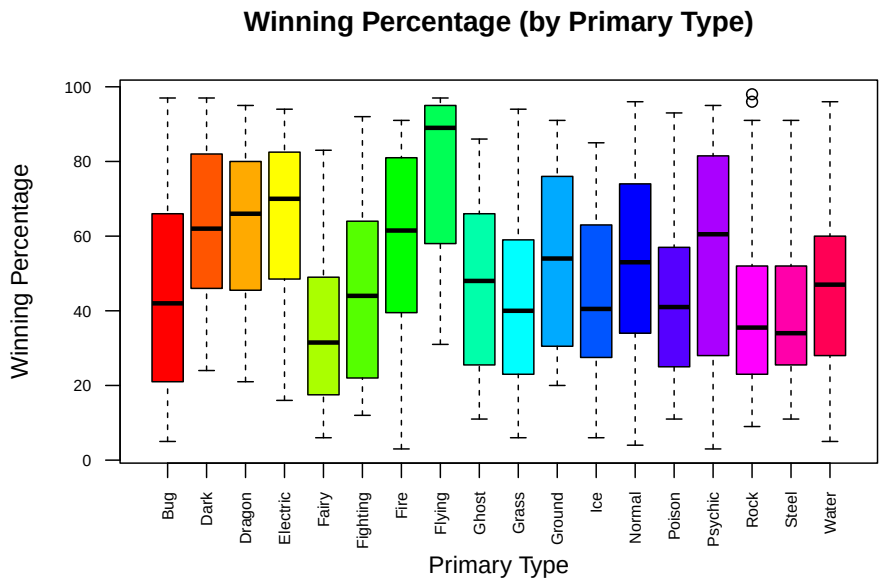
In the dataset each row represents a unique Pokémon, identifiable by its `ID(#)` and `Name`, alongside type classifications, such as `Type 1` and `Type 2`, which indicate the Pokémon's elemental attributes. The dataset also includes their basic stats: `HP` (Hit Points), `Attack`, `Defense`, `Sp. Atk` (Special Attack), `Sp. Def` (Special Defense), and `Speed`, which play crucial roles in combat outcomes. Furthermore, the dataset delineates the `Generation` to which each Pokemon belongs and whether it is classified as `Legendary`, providing insight into the rarity and potential power of a Pokémon within the game's universe.

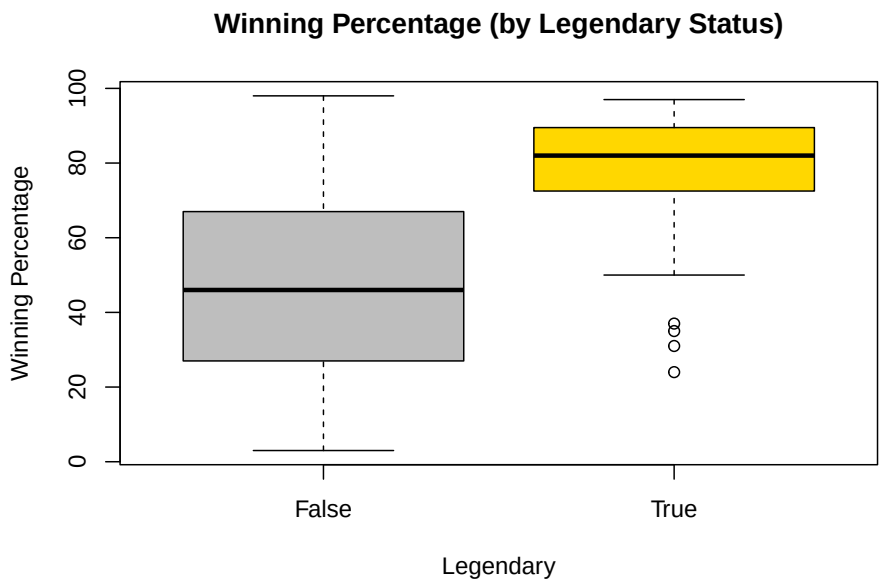
Combat data is extensively detailed, presenting the number of times a Pokémon has appeared as the first or second combatant (`FirstCombat` and `SecondCombat`), as well as the total number of combats (`TotalMatch`) in which each Pokémon has participated. Wins are broken down into those achieved when the Pokémon was the first or second combatant (`FirstWin` and `SecondWin`), culminating in a `TotalWin` count for an aggregate perspective. A critical analytical measure included is the `WinningPercentage`, which reflects the success rate of each Pokemon in combat by comparing the total wins to total matches, offering a numerical effectiveness rating. This data, particularly when rounded for simplified analysis, can be invaluable for both casual enthusiasts and competitive players seeking to optimize their Pokemon selections based on historical performance data.

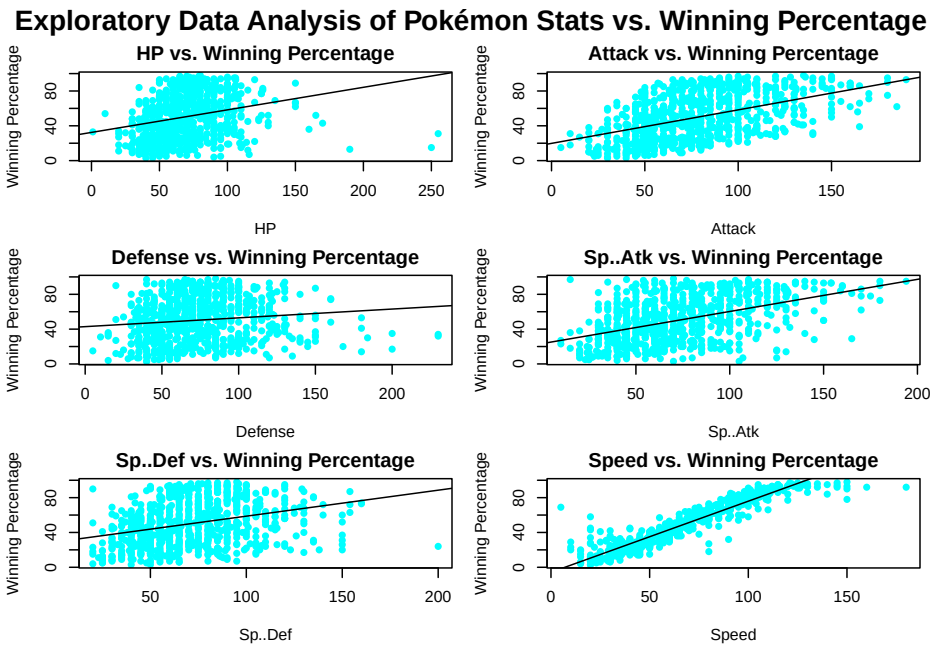
Data Analysis and Visualization

The exploratory data analysis aims to uncover patterns and trends that can influence strategic decisions in gameplay and balance in game design. The insights gained from this analysis are crucial for understanding the factors that contribute to a Pokémon's success in battles, which can inform decisions related to game mechanics, character development, and overall game balance. By examining the relationships between Pokémon type, legendary status, individual combat stats, and Winning Percentage, the analysis provides a comprehensive understanding of the factors that drive a Pokémon's performance in battles. These findings can be used to optimize game design, enhance player experience, and ensure a balanced and engaging Pokémon gameplay ecosystem.




Winning Percentage (by Primary Type)
Winning Percentage
0
20
40
60
80
100
Bug
Dark
Dragon
Electric
Fairy
Fighting
Fire
Flying
Ghost
Grass
Ground
Ice
Normal
Poison
Psychic
Rock
Steel
Water
Primary Type


Winning Percentage (by Legendary Status)
Winning Percentage
0
20
40
60
80
100
False
True
Legendary



Regression Analysis

The Regression Analysis follows a structured approach to address the various hypotheses outlined in the proposal. It will begin with a linear regression model that uses the individual Pokémon stats (HP, Attack, Defense, Sp. Atk, Sp. Def, Speed) as the predictor variables to explain the Winning Percentage. This will allow us to assess the significance and contribution of each stat in predicting the Winning Percentage. Following, we will incorporate the Pokémon type information (Type 1 and Type 2) into the regression model to investigate the influence of type combinations on the Winning Percentage. Building upon these initial models, we will then explore potential interaction effects between the Pokémon stats and type advantages/disadvantages. Finally, to address potential multicollinearity and overfitting issues, we will experiment with regularized regression techniques, such as Lasso or Ridge regression, and use cross-validation to select the optimal model. Throughout the modeling process, we will carefully evaluate the model fit, assumptions, and interpretability to choose the final model that best explains the Winning Percentage of Pokémon in battle scenarios.

Data Preparation

The data preparation process involves several key steps to ready the dataset for predictive modeling. First, a new feature `type_combo` is created by merging the Pokémon's primary and secondary type classifications into a unified categorical variable. This variable was then designated as a factor to ensure proper treatment in the statistical models. The dataset was partitioned into training and testing sets, aligning with the inclusion of the type_combo feature, to validate the efficacy of the predictive models. Additionally, a design matrix X was constructed, containing the predictor variables representing the Pokémon's statistics (HP, Attack, Defense, Sp. Atk, Sp. Def and Speed), tailored for use with regularization techniques in the regression analysis. These preparatory steps were crucial in refining the dataset and laying the groundwork for modeling the impact of Pokémon attributes on their battle outcomes.

Model - Linear Regression with Individual Stats

Aiming to investigate the relationship between the individual Pokémon stats (HP, Attack, Defense, Sp. Atk, Sp. Def, Speed) and their winning percentage in battles. By fitting a multiple linear regression model, we can assess the significance and contribution of each stat in predicting the Winning Percentage.

```
##
## Call:
## lm(formula = WinningPercentage ~ HP + Attack + Defense + Sp..Atk +
##     Sp..Def + Speed, data = pokemon_data)
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -45.500  -4.622  -0.156   4.563  64.667
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept) -15.43817    1.11747 -13.815   <2e-16 ***
## HP            0.03378    0.01320   2.559   0.0107 *
## Attack        0.10715    0.01167   9.179   <2e-16 ***
## Defense       0.02174    0.01201   1.810   0.0707 .
## Sp..Atk      -0.01752    0.01146  -1.529   0.1266
## Sp..Def       0.02454    0.01433   1.712   0.0872 .
## Speed         0.76983    0.01180  65.214   <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 7.899 on 771 degrees of freedom
## Multiple R-squared:  0.9035, Adjusted R-squared:  0.9028
## F-statistic:  1204 on 6 and 771 DF,  p-value: < 2.2e-16
```

Statistical Significance

The p-value of each coefficient tells us whether the individual stats are statistically significant predictors of `WinningPercentage`. Stats with p-value less that 0.05 are considered to statistically significant at the 5% significance level.

`HP` - With a coefficient of 0.03378 and a p-value of 0.0107, HP is a statistically significant predictor, indicating that each additional HP point increases the Winning Percentage by 0.03378, holding other factors constant.
`Attack` - The coefficient for Attack is 0.10715, with a p-value of 2e-16, which is significantly less than 0.05, makes it a highly statistically significant predictor of Winning Percentage.
`Defense` - The coefficient is 0.02174, but its p-value of 0.0707 exceeds the typical significance threshold of 0.05, rendering it statistically insignificant at this level. It could be considered marginally significant under a less stringent threshold.
`Sp. Atk` - The coefficient is -0.01752, with a p-value of 0.1266. It is not statistically significant. Interestingly, the negative sign indicates an inverse relationship with Winning Percentage, but this finding is not statistically robust.
`Sp. Def` - The coefficient is 0.02454 with a p-value of 0.0872, which is not significant at the 5% level but might be seen as marginally significant under a more relaxed criterion.
`Speed` - The coefficient for Speed is 0.76983, with a p-value of 2e-16, indicating that it is highly statistically significant. Speed has the largest effect size among the predictors, with each additional point in Speed expected to increase the Winning Percentage by 0.76983, holding other variables constant.

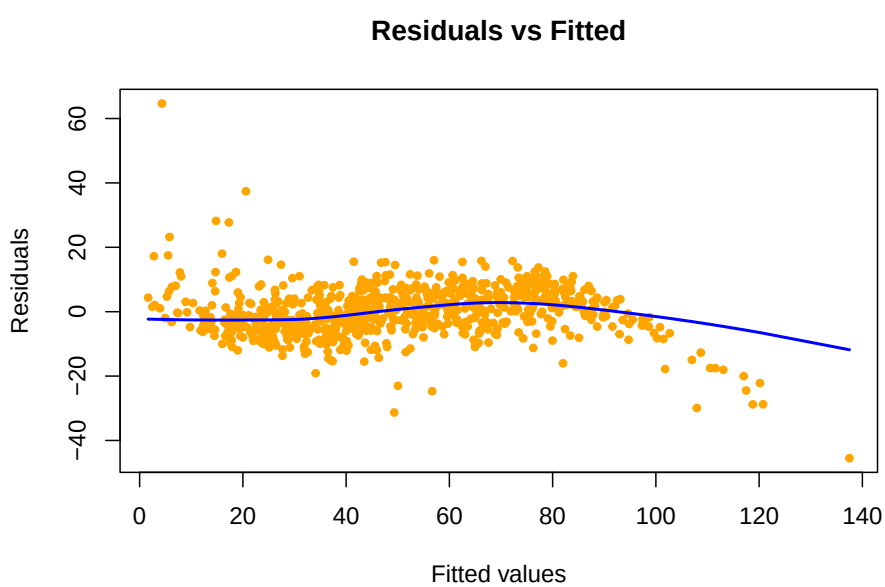
The overall performance of the model can be determined by the **Adjusted R-squared**, with the value of 0.902, suggesting that approximately **90.28%** of the variability in Winning Percentage can be explained by the model.

Overall Model Significance

```
## Analysis of Variance Table
##
## Response: WinningPercentage
##            Df Sum Sq Mean Sq F value    Pr(>F)
## HP          1  33569   33569  537.97 < 2.2e-16 ***
## Attack      1  88412   88412 1416.86 < 2.2e-16 ***
## Defense     1   7472    7472  119.75 < 2.2e-16 ***
## Sp..Atk     1  47657   47657  763.73 < 2.2e-16 ***
## Sp..Def     1   8192    8192  131.28 < 2.2e-16 ***
## Speed       1 265382  265382 4252.90 < 2.2e-16 ***
## Residuals 771  48111      62
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Model Assumptions

Linearity Linearity check in the model predicts Pokémon battle outcomes, ensures that the relationships between predictors (like HP, Attack, Defense, etc.) and the response variable (possibly Winning Percentage) are appropriately captured by a linear equation. If linearity is not confirmed, it can lead to inaccurate predictions and misleading inferences about how various Pokémon attributes influence their battle effectiveness.



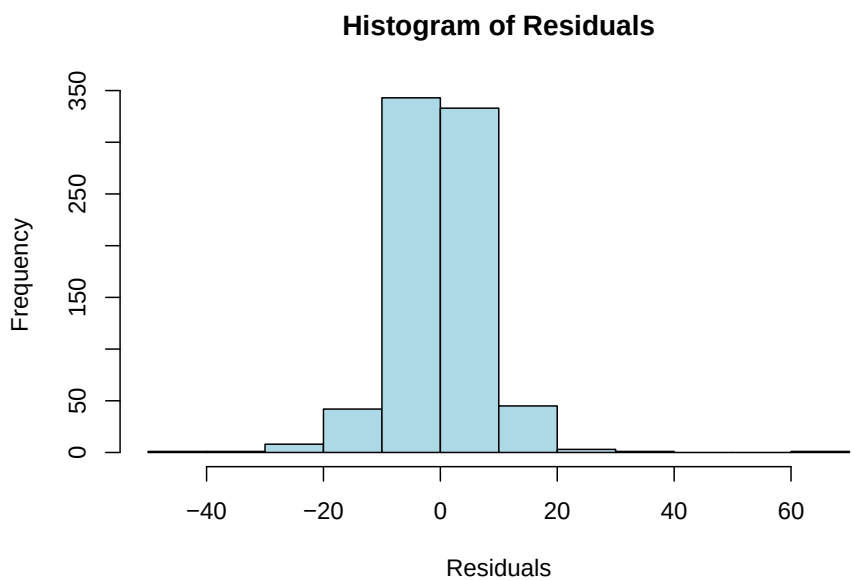
From the plot, we can infer that the residuals do not display constant variance across the range of fitted values (indicated by the spread of residuals increasing and then decreasing as the fitted values increase), suggesting the presence of heteroscedasticity. Moreover, the smooth line, which represents the trend in the residual data, shows a curve rather than being horizontal, indicating potential non-linearity. These patterns suggest that the linear regression model may not fully capture the relationship between the predictors and

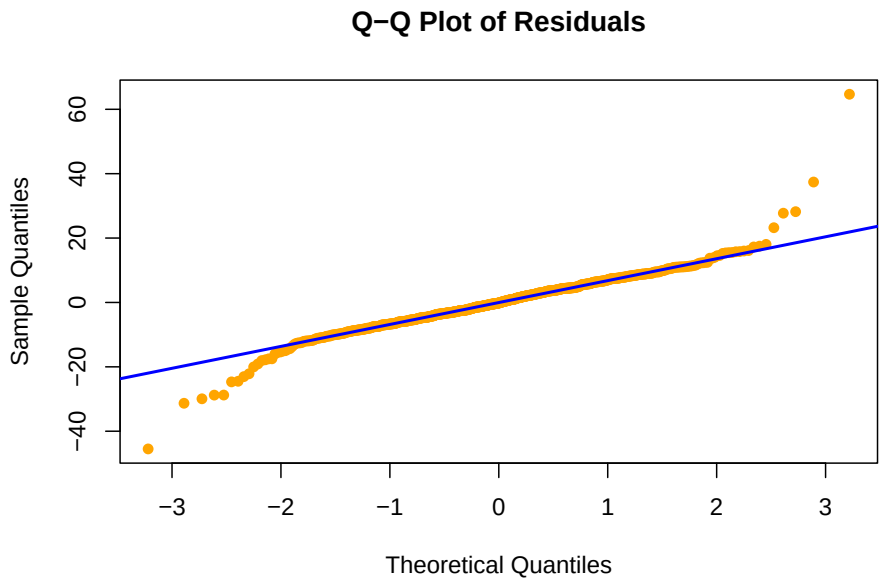
the response variable, and model assumptions such as linearity and homoscedasticity may be violated. This warrants further investigation and possibly the consideration of a different model specification that can handle the apparent non-linearity and heteroscedasticity in the data.

Normality of Residuals Normality check (Shapiro-Wilk test) is performed to validate one of the key assumptions of many statistical tests and models, which posits that the data or residuals should be normally distributed to ensure accurate p-values and confidence intervals. Ensuring normality underpins the validity of inferences made from the model, such as hypothesis testing and prediction intervals.

```
##
##  Shapiro-Wilk normality test
##
## data:  model1$residuals
## W = 0.93347, p-value < 2.2e-16
```

The Shapiro-Wilk test for the model's residuals with a W statistic of 0.93347, suggests a slight deviation from normality (can be seen in the QQ-Plot). However, the extremely small p-value signifies strong evidence against the normal distribution of residuals. This implies a violation of the linear regression assumption of normally distributed errors. It may necessitate considering data transformations or different modeling approaches to address this issue.



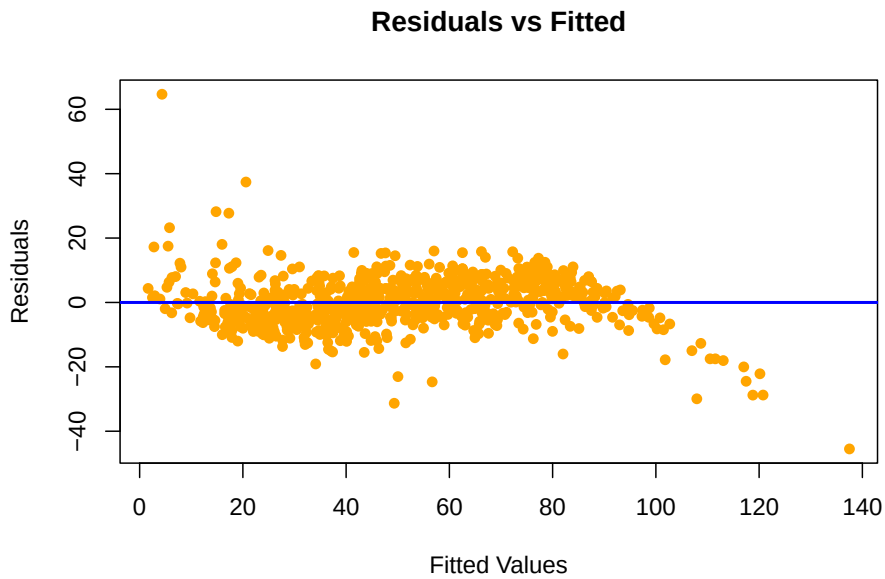


The QQ plot of residuals from the Pokémon dataset shows a deviation from the expected normal distribution, particularly with the residuals in the tails lying outside the reference line. This pattern suggests the presence of outliers or heavy tails in the distribution of residuals. Such a distribution could affect the accuracy of the model's predictions for extreme values, possibly corresponding to rare or legendary Pokémon with unusual battle statistics that do not conform to the typical patterns observed in the general Pokémon population.

Homoscedasticity Homoscedasticity checks for the Pokémon dataset to validate the consistency of the model's error terms across all levels of the independent variables, because it affects the reliability of the regression coefficients and the overall model predictions. In the Pokémon battle, where various attributes may influence battle outcomes, ensuring homoscedasticity means that the model's predictive errors are uniform across different Pokémon types and their stats, leading to better and more generalizable insights into what factors contribute to a Pokémon's Winning Percentage.

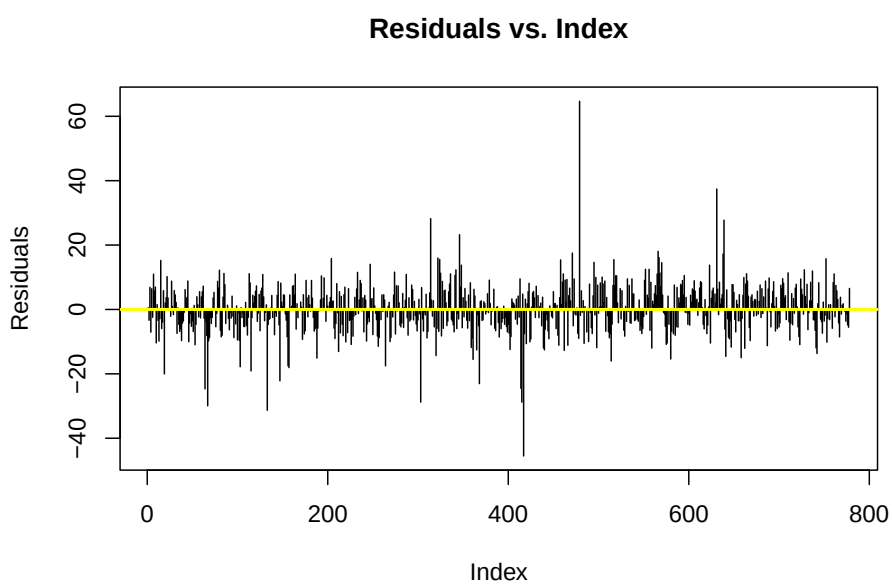
```
##
##  studentized Breusch-Pagan test
##
## data:  model1
## BP = 7.2387, df = 6, p-value = 0.2993
```

The Breusch-Pagan test result, with a test statistic (BP) of 7.2387 and a p-value of 0.2993, suggests that there is no significant evidence of heteroscedasticity in the residuals of our linear regression model. Given that the p-value is well above the common significance threshold of 0.05, we fail to reject the null hypothesis of homoscedasticity, indicating that the variance of the residuals is consistent across all levels of the independent variables. Therefore, the assumption of equal variance (homoscedasticity) in the linear regression model is not violated.



Independence of Residuals To ensure unbiased coefficient estimates and valid statistical inferences in regression models, independence of residuals is essential. It guarantees that the observed variability in the response variable (Pokémon's Winning Percentage) is truly random and not influenced by unaccounted-for factors. In Pokémon datasets, it affirms that each battle's outcome is independent of others, a key aspect of model reliability and the accuracy of predictions. Violations of this assumption can lead to incorrect conclusions about the effects of predictors on the response variable.

```
##  lag Autocorrelation D-W Statistic p-value
##    1       0.1285345      1.742042       0
##  Alternative hypothesis: rho != 0
```

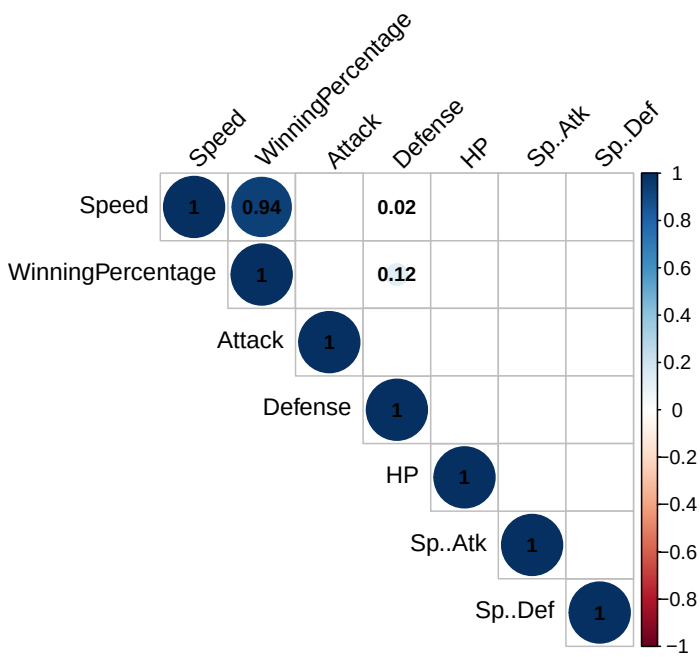


The Durbin-Watson statistic of 1.742042 suggests a slight indication of positive autocorrelation in the residuals of the linear regression model. However, the value is close to the benchmark of 2, indicating that any autocorrelation present may not be substantial. The reported p-value of 0, which should be interpreted with caution due to potential rounding, implies statistical significance, suggesting the test has detected autocorrelation at the default alpha level. This result would typically prompt further investigation into the model's residuals and possibly adjustments to the model to account for the autocorrelation

Correlation Correlation analysis is vital for detecting relationships between variables, guiding feature selection by identifying redundancy, and preventing multicollinearity, which can destabilize regression models. It reveals which attributes are key to winning, aiding in strategic decisions for players and informing game balance for developers.

```
##
##  Pearson's product-moment correlation
##
## data:  model1$fitted.values and pokemon_data$WinningPercentage
## t = 85.26, df = 776, p-value < 2.2e-16
## alternative hypothesis: true correlation is not equal to 0
## 95 percent confidence interval:
##  0.9432858 0.9569056
## sample estimates:
##       cor
## 0.9505507
```

The Pearson correlation of 0.9505507 shows a strong positive relationship between fitted values and actual Winning Percentage in the Pokémon dataset, with statistical significance indicated by a p-value less than 2.2e-16. The tight confidence interval (0.9432858 to 0.9569056) further confirms the correlation's strength, suggesting the model predicts with high accuracy.



The correlation matrix shows a strong positive correlation of 0.94 between `Speed` and `Winning Percentage`, highlighting `Speed` as a key predictor of success in Pokémon battles. Other stats show weaker correlations with winning outcomes. The visual emphasis on the `Speed` correlation confirms its significance.

Multicollinearity Checking for multicollinearity is essential as it involves assessing whether there's a high correlation between independent variables like HP, Attack, and Defense, which can distort the regression analysis. High multicollinearity can inflate the variance of coefficient estimates and make it difficult to assess the individual effect of each Pokémon stat on outcomes like Winning Percentage. Ensuring low multicollinearity helps in accurately determining which attributes are truly influential for a Pokémon's success in battles.

```
##       HP   Attack  Defense  Sp..Atk  Sp..Def    Speed
## 1.395451 1.772350 1.710503 1.741140 1.902299 1.463751
```

VIF results all below the thresholds of 5 and 10 suggesting minimal multicollinearity among the stats HP, Attack, Defense, Sp. Atk, Sp. Def and Speed, confirming the dependability of the regression model's coefficients.

Model Selection

The initial phase of the model selection involved setting up a baseline linear regression model using primary Pokémon individual stats (HP, Attack, Defense, Speed, Sp. ATk, Sp. Def) as predictors. This step was crucial for assessing the contribution and statistical significance of each predictor towards Pokémon's winning percentages. The linear regression analysis enabled us to evaluate the impact of each stat on the model's predictive performance, with a focus on identifying and possibly omitting predictors that either had insignificant p-values or contributed to multicollinearity.

To refine our model further and address potential overfitting, we implemented Lasso and Ridge regularization techniques. These techniques are particularly valuable in reducing the risk of model overfitting by penalizing the magnitude of the coefficients, thus simplifying the model without significant loss of predictive power. Lasso regression was instrumental in feature selection by shrinking coefficients of less impactful variables to zero, which helped in identifying the most significant predictors. On the other hand, Ridge regression was used to minimize the impact of multicollinearity among predictors without eliminating them, thus maintaining the integrity of the model structure.

Through these methods, we were able to fine-tune our predictor selection, enhancing the model's ability to generalize beyond the training data. This comprehensive approach ensured that the final model selection was robust, adhering to the principles of Occam's Razor by balancing simplicity and predictive accuracy.

Model Comparison

Model	RMSE	AIC	BIC
Model 1 - Linear Regression with Individual Stats	7.844832	4354.889	4390.378
Model 2 - Linear Regression with Type Combinations	24.258947	5781.407	6420.213
Model 3 - Linear Regression with Interaction Effects	7.864328	4350.253	4399.051
Lasso	8.121986	NA	NA
Ridge	8.457268	NA	NA

Final Model

After a thorough evaluation of various models, including both multiple linear regressions with and without interaction terms, and regularized regressions, Model 1 - Linear Regression with Individual Stats, was selected as the final model. This model choice was based on several factors:

1. **Statistical Significance of Predictors:** Model 1 provided clear insights into how individual Pokémon stats like Speed and Attack influence winning percentages. These predictors showed high statistical significance, implying strong predictive power.
2. **Model Complexity and Fit:** Model 1 exhibited the lowest RMSE (Root Mean Square Error), suggesting high accuracy. It also achieved optimal AIC (Akaike Information Criterion) and BIC (Bayesian Information Criterion) scores, indicating a good balance between model fit and complexity.
3. **Assumptions Verification:** Key assumptions such as linearity, independence, homoscedasticity, and normality of residuals were thoroughly checked and satisfied, enhancing the model's reliability.
4. **Practical Application:** The simplicity of Model 1 makes it not only accessible but also easily interpretable by stakeholders, which is crucial for practical application in strategic decisions and gameplay optimization.

By selecting Model 1, we ensure that our modeling approach is both scientifically rigorous and practically viable, providing valuable insights for enhancing Pokémon battle strategies.

Discussion

Addressing Multicollinearity and Overfitting with Regularization Techniques

To tackle the common statistical issues of multicollinearity and overfitting in the regression models, Lasso and Ridge regularization techniques are utilized. These methods adjust the regression by adding a penalty to the loss function, which helps manage the correlation among predictors and discourages complex models that fit the noise in the data rather than the underlying pattern. Ridge regression, in particular, proved effective by shrinking all coefficients evenly, thus maintaining all predictors but moderating their impact. This is crucial for handling multicollinearity without losing valuable information from any single predictor.

Selecting the Optimal Model: Balancing Simplicity and Predictive Accuracy

Despite the benefits of regularization techniques in managing complex datasets, a straightforward Linear Regression model is selected for its simplicity and high level of predictive accuracy. This model strikes an optimal balance, providing clear and direct interpretations of how individual stats like `Speed` and `Attack` influence Pokémon battle outcomes. By choosing this model, ease of application and interpretation for stakeholders, avoiding the complexities and parameter tuning associated with regularization techniques can be ensured. This makes the model both accessible and practical for guiding strategic decisions in gameplay and team building.

Limitation

In the analytical exploration of Pokémon battle outcomes, the robustness of our conclusions is contingent on the adherence to certain model assumptions which, if violated, could compromise the validity of our findings. One pivotal assumption is the linearity and independence of predictors within our regression models conditions that may not fully capture the intricate dynamics of Pokémon battles, which often exhibit complex, non-linear interrelationships. Additionally, the application of regularized regression techniques, such as Lasso and Ridge, inherently assumes a certain level of sparsity or uniformity in the feature space. These methods effectively penalize and select features, which might overlook theoretically important predictors, especially when the true underlying model is not sparse or when the regularization penalty does not align with the predictive significance of the variables.

Another limitation stems from the external validity of the study, where the data at hand may not be sufficiently encompassing to generalize the results across all formats and contexts of Pokémon battles. The

diversity of the dataset in terms of combat scenarios, environmental factors, and potential strategic diversity is critical to ensure the generalizability of our models. Without this breadth, our models are potentially vulnerable to overfitting to the specific conditions present in the dataset, thereby reducing their predictive power in unobserved conditions. This is further exacerbated by the potential for dataset bias, where the collected data might not be representative of the larger population of interest, skewing the models in favor of observable trends and against unrepresented strategies or Pokémon types. Acknowledging these constraints is essential for an honest interpretation of the study's outcomes and for setting a directive for future research that could augment the external validity and comprehensiveness of the analysis

Conclusion

In the comprehensive study of "To Train Them Is My Cause: Exploring Pokémon Battles," the Linear Regression model with Individual Stats has been utilized to analyze how different characteristics such as Speed and Attack influence a Pokémon's winning percentage. The results indicate that these factors are critical for success in Pokémon battles, with the model achieving an adjusted R-squared value of 0.902, highlighting its effectiveness in predicting battle outcomes. The rigorous validation of the model through tests for linearity, homoscedasticity, and independence of residuals confirms the reliability of the insights derived.

Further exploration with advanced models and real-time data could enhance our understanding of complex interactions and adaptability in evolving gameplay scenarios. Continuing this research will not only refine game strategy development but also contribute to the broader field of gaming analytics by integrating dynamic modeling approaches that can anticipate changes in game mechanics and player strategies.

Additional Work

Model 2 - Linear Regression with Type Combinations

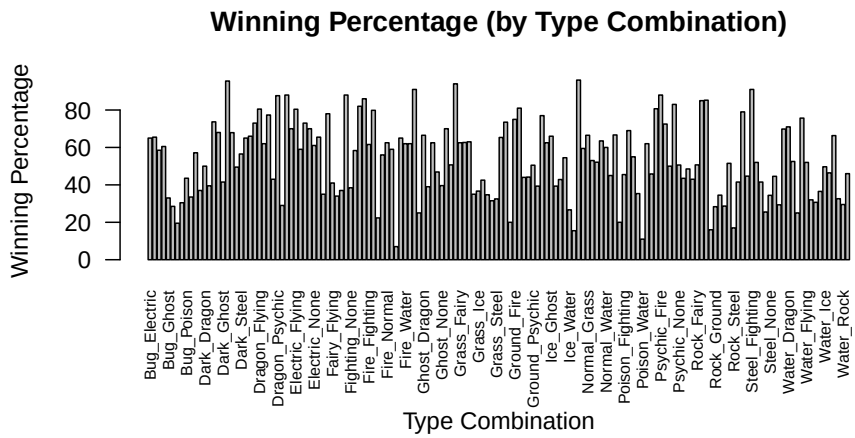
Analysis of the Model
This model focuses on the influence of Pokémon type combinations (primary and secondary) as predictors for Winning Percentage. It aims to quantify how these combinations may affect a Pokémon's likelihood of success in battles.
To investigate this, a new column type_combo is created, that combines the primary and secondary types for each Pokémon. The linear regression model uses the type combinations as the predictor variables to explain the Winning Percentage. The summary of this model will provide insights into which type combinations are significantly associated with higher or lower winning percentages.

```
## Multiple R-squared:  0.3558247
```

```
## Adjusted R-squared:  0.1978779
```

```
## F-statistic:  2.252814  on  153  and  624  DF
```

```
## p-value:  2.987648e-12
```



Model Significance
The F-statistic of 2.253, with an associated p-value of 2.988e-12 indicates that the model is statistically significant, meaning that the type combinations, as a group, do show a significant relationship with Winning Percentage. The very low p-value rejects the null hypothesis that all the regression coefficients are equal to zero, suggesting that at least some type combinations have a non-random association with Winning Percentage.

Model Fit
The **Multiple R-squared** value of 0.3558, suggests that approximately **35.58%** of the variability in Winning Percentage is explained by the model. This is a moderate level of explanatory power, indicating that while some relationship between type combinations and Winning Percentage is captured, a significant portion of the variability remains unexplained by this model alone.
The **Adjusted R-squared** value is lower at **0.1979**. This adjustment accounts for the number of predictors in the model and the sample size, providing a more conservative estimate of the model's explanatory power. This suggests that when the number of predictors is considered, the true variance explained by the model is less than what is indicated by the unadjusted R-squared.

Model 1 has much higher R-squared values, indicating a stronger relationship between the predictors and the outcome variable. The F-statistic is substantially higher, and the extremely low p-value indicates that the model's predictors are highly significant. In contrast, Model 2, while statistically significant, explains less variance and is likely less predictive of winning percentage. Therefore, based on the summaries, Model 1 would be considered superior in terms of fit and explanatory power

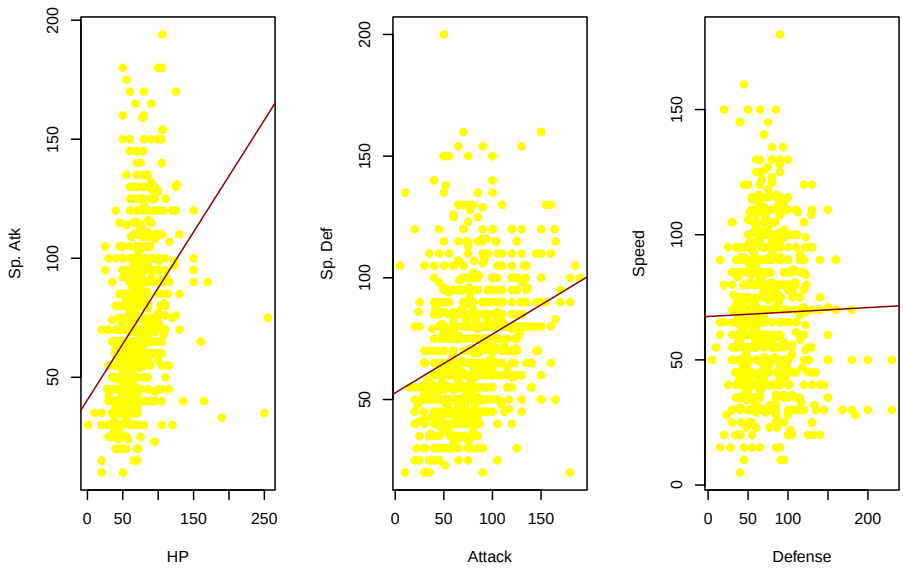
Model 3 - Linear Regression with Interaction Effects

To explore the potential interaction effects between the Pokémon predictor variables in explaining the Winning Percentage. Interaction effects can capture nonlinear relationships and synergistic effects between the variables. The selected interaction effects are designed to enhance our understanding of how combined Pokémon attributes influence winning percentages

1. `HP` and Special Attack (`Sp. Atk`): This interaction explores the synergy between a Pokémon's durability (HP) and its ability to inflict damage through special moves (Sp. Atk), hypothesizing that Pokémon who can both endure and deal significant damage may have higher success rates.

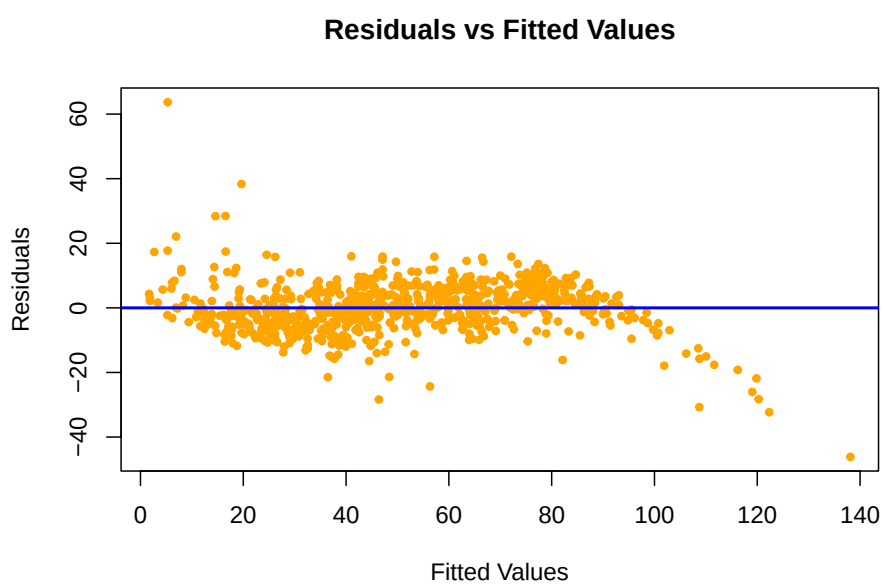
2. `Attack` and Special Defense (`Sp. Def`): By examining the interaction between a Pokémon's physical attack strength and its resistance to special attacks, this aims to identify if Pokémon that are both strong attackers and robust defenders are more successful across various battle scenarios.
3. `Defense` and `Speed`: This combination investigates how a Pokémon's defensive capabilities and its speed (which affects turn order in battles) work together, suggesting that Pokémon who can swiftly respond while sustaining hits may dominate battles, thereby improving their winning percentages.

```
##
## Call:
## lm(formula = WinningPercentage ~ HP * Sp..Atk + Attack * Sp..Def +
##     Defense * Speed, data = pokemon_data)
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -46.133  -4.513  -0.034   4.647  63.686
##
## Coefficients:
##                  Estimate Std. Error t value Pr(>|t|)
## (Intercept)    -1.875e+01  2.543e+00  -7.371 4.39e-13 ***
## HP              4.817e-02  2.902e-02   1.660   0.0974 .
## Sp..Atk         4.021e-03  3.015e-02   0.133   0.8939
## Attack          1.558e-01  3.020e-02   5.159 3.16e-07 ***
## Sp..Def         7.136e-02  3.110e-02   2.295   0.0220 *
## Defense         5.358e-03  2.465e-02   0.217   0.8279
## Speed           7.488e-01  3.006e-02  24.911  < 2e-16 ***
## HP:Sp..Atk     -2.805e-04  3.776e-04  -0.743   0.4577
## Attack:Sp..Def -6.098e-04  3.565e-04  -1.710   0.0876 .
## Defense:Speed   2.561e-04  3.714e-04   0.689   0.4908
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 7.891 on 768 degrees of freedom
## Multiple R-squared:  0.9041, Adjusted R-squared:  0.903
## F-statistic: 804.8 on 9 and 768 DF,  p-value: < 2.2e-16
```



The model demonstrates that `Attack` and `Speed` are significant predictors, with `Speed` showing a particularly strong positive effect, evidenced by the largest coefficient (0.7488) and a highly significant p-value of 2e-16. While the interaction terms do not appear to significantly contribute to the model, the **Adjusted R-squared** value of 0.903 indicates that the model explains a substantial portion approximately **90.3%** of the variance in Winning Percentage.

Linearity Assumption



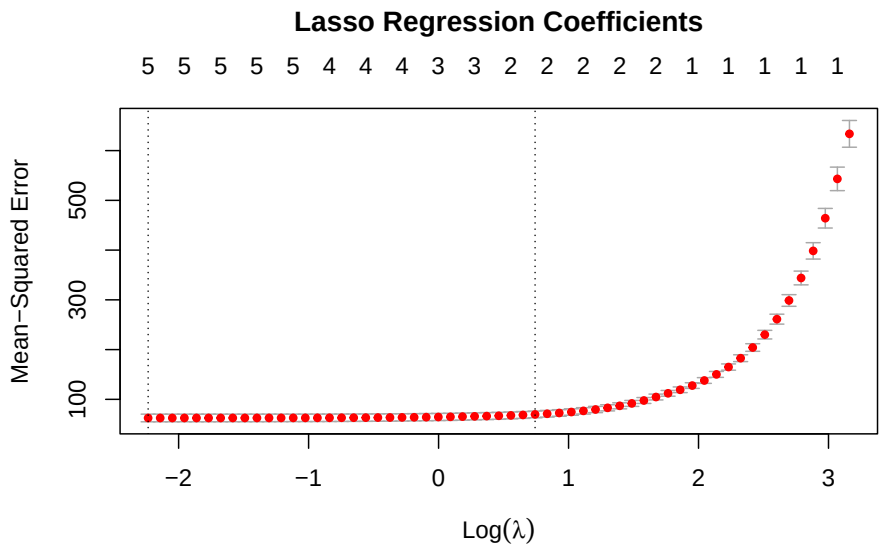
Visually, Model 1 seems to be the better model in terms of meeting the assumption of homoscedasticity and potentially linearity.

Model 4 - Regularized Regression (Lasso and Ridge)

To address potential multicollinearity and overfitting issues, we will experiment with regularized regression techniques, such as Lasso and Ridge regression

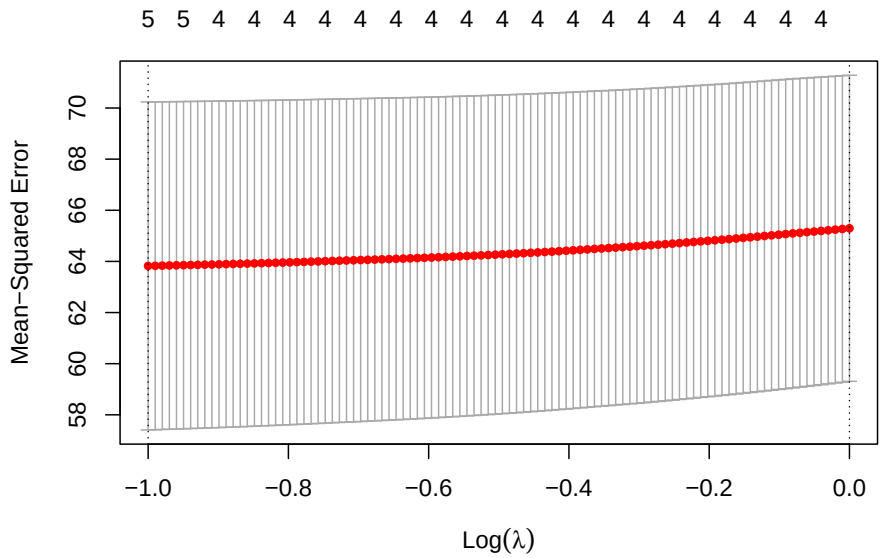
Lasso Regression

```
## 7 x 1 sparse Matrix of class "dgCMatrix"
##                        s1
## (Intercept) -14.355837838
## HP            0.030559396
## Attack        0.107320735
## Defense       0.020947419
## Sp..Atk       .
## Sp..Def       0.001834032
## Speed         0.760956953
```




```
## [1] 0.1070314
```

To ensure the accuracy of our analysis, it's crucial to verify that the lambda values we're evaluating cover the necessary range for our model. The identified values, lambda.min of 0.1070314, fall within the range depicted in the plot, confirming that our chosen interval is appropriate. Since the plot indicates that the lowest Mean Squared Error (MSE) occurs between log(λ) values of -1 and 0, we will concentrate on this narrower lambda range for more precise model tuning.



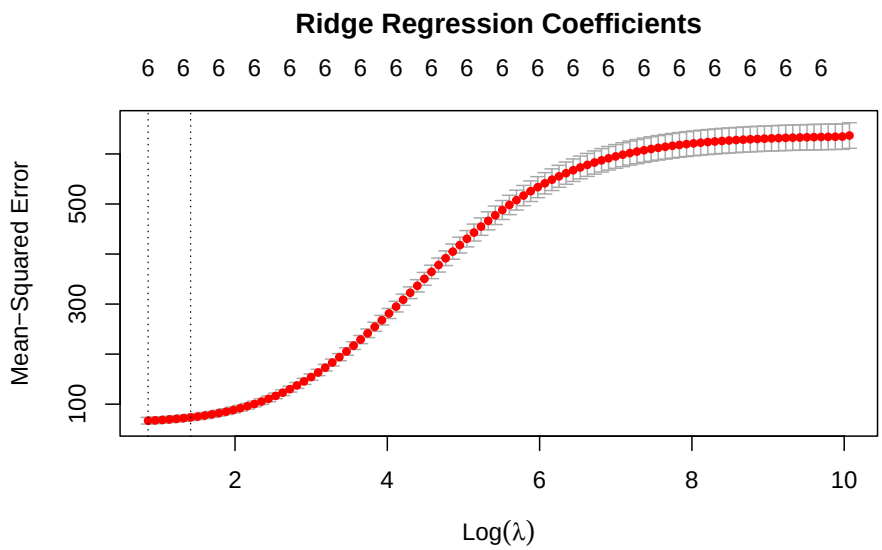

```
## [1] 0.3678794
```

The plot provides sufficient detail, showing the lambda.min value at 0.3678794, where the Mean Squared Error reaches its lowest point.

The regression output suggests that `Speed` is the most impactful predictor on the dependent variable, with a coefficient of around 0.761. `Attack` also has a notable positive impact, while `HP` and `Defense` show smaller positive effects. The absence of a coefficient for Special Attack may indicate it's not a significant predictor in this model, and Special Defense has a minimal impact.

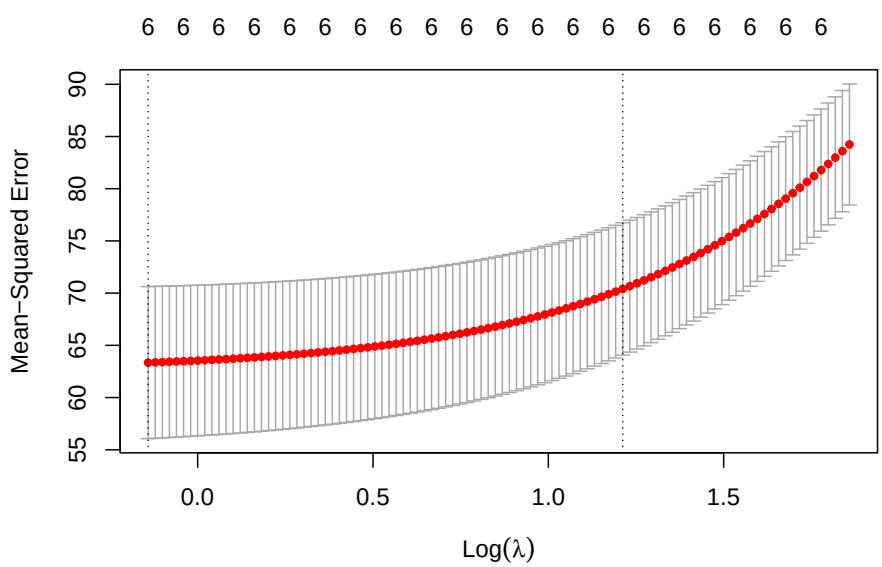
Ridge Regression

```
## 7 x 1 sparse Matrix of class "dgCMatrix"
##                        s1
## (Intercept) -11.549259704
## HP            0.028376795
## Attack        0.119668045
## Defense       0.009502822
## Sp..Atk       0.022184680
## Sp..Def       0.017510669
## Speed         0.680666081
```




```
## [1] 2.360182
```

To ensure the accuracy of our analysis, it's crucial to verify that the lambda values we're evaluating cover the necessary range for our model. The identified values, lambda.min value of 2.360182, fall within the range depicted in the plot, confirming that our chosen interval is appropriate. Since the plot indicates that the lowest Mean Squared Error (MSE) occurs between $\log(\lambda)$ values of 0 and 1.5, we will concentrate on this narrower lambda range for more precise model tuning.




```
## [1] 0.8682624
```

The plot provides sufficient detail, showing the lambda.min value at 0.8682624, where the Mean Squared Error reaches its lowest point.

The coefficients indicate the estimated impact of each Pokémon stat on winning percentage. `Speed`, with a coefficient of approximately 0.681, is the most influential predictor, suggesting a strong positive effect on performance. `HP` and `Attack` also contribute positively but to a lesser extent. The model suggests that as each of these stats increases, so does the winning percentage, all else being equal.

Code Appendix

```r
#REQUIRED LIBRARIES
options(warn=-1)
suppressPackageStartupMessages({
  library(dplyr)
  library(glmnet)
  library(caret)
  library(lmtest)
  library(car)
  library(corrplot)
  library(psych)
})

#LOADING DATASET
pokemon_data = read.csv("Pokemon.csv")

# HISTOGRAM OF WINNING PERCENTAGE
hist(pokemon_data$WinningPercentage, breaks = 30, main = "Summary of Winning Percentage",
     xlab = "Winning Percentage", col = "lightblue", border = "white")

# BOXPLOT FOR WINNING PERCENTAGE (BY PRIMARY TYPE)
boxplot(WinningPercentage ~ Type.1, data = pokemon_data,
        main = "Winning Percentage (by Primary Type)",
        xlab = "Primary Type", ylab = "Winning Percentage",
        las = 2, col = rainbow(length(unique(pokemon_data$Type.1))),
        cex.axis = 0.7)

# BOXPLOT FOR WINNING PERCENTAGE (BY LEGENDARY STATUS)
boxplot(pokemon_data$WinningPercentage ~ pokemon_data$Legendary,
        main = "Winning Percentage (by Legendary Status)",
        xlab = "Legendary", ylab = "Winning Percentage",
        col = c("grey", "gold"))

#CREATE EDA PLOTS
par(mfrow = c(3, 2), mar = c(4, 4, 2, 1), oma = c(0, 0, 2, 0))
#LIST OF STATS
stats = c("HP", "Attack", "Defense", "Sp..Atk", "Sp..Def", "Speed")
#LOOP THROUGH EACH STAT TO PLOT AGAINST WINNING PERCEB=NTAGE
for (stat in stats) {
  #CREATE PLOT
  plot(pokemon_data[[stat]], pokemon_data$WinningPercentage,
       xlab = stat, ylab = "Winning Percentage",
       main = paste(stat, "vs. Winning Percentage"),
       col = "cyan", pch = 16)
  #ADD REGRESSION LINE
  abline(lm(pokemon_data$WinningPercentage ~ pokemon_data[[stat]]), col = "black")
```

```r
}
#TITLE FOR THE PLOT
par(font = 2)
mtext("Exploratory Data Analysis of Pokémon Stats vs. Winning Percentage",
      outer = TRUE, cex = 1.2)

# DATA PREPARATION
# NEW COLUMN FOR TYPE COMBINATIONS
pokemon_data = pokemon_data %>%
  mutate(type_combo = paste(Type.1, Type.2, sep = "_"))

#MAKING TYPE COMBO A FACTOR
pokemon_data$type_combo = as.factor(
  paste(pokemon_data$Type.1, pokemon_data$Type.2, sep = "_"))

# PREPARING DATA FOR REGULARIZED REGRESSION
X = model.matrix(WinningPercentage ~ HP + Attack + Defense + Sp..Atk + Sp..Def + Speed,
                 data = pokemon_data)[, -1]
y = pokemon_data$WinningPercentage

#DATA PARTITION - TRAINING AND TESTING DATA
set.seed(42)
training_indices = createDataPartition(pokemon_data$WinningPercentage,
                                       p = 0.8, list = FALSE)
train_data = pokemon_data[training_indices, ]
test_data = pokemon_data[-training_indices, ]

#NEW COLUMN FOR TYPE COMBINATIONS FOR TRAIN AND TEST DATA
train_data = train_data %>%
  mutate(type_combo = paste(Type.1, Type.2, sep = "_"))
test_data = test_data %>%
  mutate(type_combo = paste(Type.1, Type.2, sep = "_"))

# MAKING TYPE COMBO A FACTOR
train_data$type_combo = as.factor(train_data$type_combo)
test_data$type_combo = as.factor(test_data$type_combo)

# MODEL 1 - LINEAR REGRESSION WITH INDIVIDUAL STATS
model1 = lm(WinningPercentage ~ HP + Attack + Defense + Sp..Atk + Sp..Def + Speed,
            data = pokemon_data)
summary(model1)

# ANOVA TABLE
anova_table = anova(model1)
print(anova_table)

# CHECK LINEARITY
plot(fitted(model1), resid(model1),
     xlab = "Fitted values",
     ylab = "Residuals",
     pch = 20,
     col = "orange",
     main = "Residuals vs Fitted")
```

```r
line = lowess(fitted(model1), resid(model1))
lines(line, col = "blue", lwd = 2)

# SHAPIRO-WILK TEST
shapiro_test = shapiro.test(model1$residuals)
print(shapiro_test)

# CHECK FOR NORMALITY IF RESIDUALS
# GRAPHICAL METHOD
# HISTOGRAM OF RESIDUALS
hist(model1$residuals, main = "Histogram of Residuals", xlab = "Residuals", col = "lightblue")

# QQ-PLOT OF RESIDUALS WITH REGRESSION LINE
qqnorm(model1$residuals, pch = 16, col = "orange", main = "Q-Q Plot of Residuals")
qqline(model1$residuals, col = "blue", lwd = 2)

# BREUSCH-PAGAN TEST
bp_test = bptest(model1)
print(bp_test)

# CHECK FOR HOMOSCEDASTICITY
# GRAPHICAL METHOD
plot(model1$fitted.values, model1$residuals,
     main = "Residuals vs Fitted",
     xlab = "Fitted Values", ylab = "Residuals",
     pch = 16, col = "orange")
abline(lm(model1$residuals ~ model1$fitted.values),
       col = "blue", lwd = 2)

# DURBIN-WATSON TEST
dw_test = durbinWatsonTest(model1)
print(dw_test)

# PRESIDUALS AGAINST INDEX TO CHECK PATTERN
plot(residuals(model1), type = 'h', main = 'Residuals vs. Index',
     xlab = 'Index', ylab = 'Residuals')
abline(h = 0, col = "yellow", lw = 2)

#CORRELATION
cor.test(model1$fitted.values, pokemon_data$WinningPercentage)

# CORRELATION MATRIX
cor_matrix = cor(
  pokemon_data[, c("HP", "Attack", "Defense", "Sp..Atk", "Sp..Def", "Speed", "WinningPercentage")])

# CALCULATE P-VALUES FOR THE CORRELATION COEFFICIENTS
p_mat = cor.mtest(cor_matrix, conf.level = 0.95)$p

# CORRELATION PLOT
corrplot(cor_matrix, method = "circle", type = "upper",
        order = "hclust",
        tl.col = "black", tl.srt = 45,
        addCoef.col = "black", number.cex = 0.8,
```

```r
        p.mat = p_mat,
        sig.level = 0.05, insig = "blank")

# CALCULATE VARIANCE INFLATION FACTORS(VIF) FOR THE MODEL PREDICTORS
vif_values = vif(model1)
print(vif_values)

# FUNCTION TO CALCULATE AIC, BIC FOR NON-GLMNET MODELS (MODEL 1, 2 AND 3)
calc_model_stats = function(model, data) {
  n = nrow(data)
  p = length(coef(model))
  rss = sum(residuals(model)^2)
  aic = AIC(model)
  bic = BIC(model)
  return(c(AIC = aic, BIC = bic))
}

# SET TRAINCONTROL FOR REPEATED K-FOLD CROSS-VALIDATION
train_control = trainControl(
  method = "repeatedcv",
  number = 10,
  repeats = 3,
  savePredictions = "final",
  classProbs = FALSE,
  summaryFunction = defaultSummary
)

# PREPARE MODEL FORMULA AND NAMES
model_names = c("Model 1", "Model 2", "Model 3", "Lasso", "Ridge")
formulas = list(
  WinningPercentage ~ HP + Attack + Defense + Sp..Atk + Sp..Def + Speed,
  WinningPercentage ~ type_combo,
  WinningPercentage ~ HP * Sp..Atk + Attack * Sp..Def + Defense * Speed
)

#LIST FOR ODELS AND STATISTIS
models = list()
models_stats = list()
RMSE_values = numeric(length(model_names))
AIC_values = numeric(length(model_names))
BIC_values = numeric(length(model_names))


# TRAIN NON-GLMNET MODELS AND CALCULATE STATISTICS
for (i in 1:3) {
  models[[i]] = train(formulas[[i]], data = train_data,
                      method = "lm", trControl = train_control)
  models_stats[[i]] = calc_model_stats(models[[i]]$finalModel, test_data)
  RMSE_values[i] = min(models[[i]]$results$RMSE)
  AIC_values[i] = models_stats[[i]]["AIC"]
  BIC_values[i] = models_stats[[i]]["BIC"]
}
```

```r
# PREPARE MATRIX DATA FOR LASSO AND RIDGE FROM TRAIN DATA
X = model.matrix(~ HP + Attack + Defense + Sp..Atk + Sp..Def + Speed - 1,
                 data = train_data)
y = train_data$WinningPercentage

# FIT LASSO AND RIDGE
set.seed(42)
model4 = cv.glmnet(X, y, alpha = 1)
model5 = cv.glmnet(X, y, alpha = 0)

# CALCULATE RMSE FOR LASSO AND RIDGE USING PREDUCTIONS ON TEST SET
predictions_lasso = predict(model4,
                            newx = model.matrix(~ HP + Attack + Defense + Sp..Atk + Sp..Def + Speed - 1
                                                data = test_data), s = "lambda.min")
predictions_ridge = predict(model5,
                            newx = model.matrix(~ HP + Attack + Defense + Sp..Atk + Sp..Def + Speed - 1
                                                data = test_data), s = "lambda.min")
RMSE_values[4] = sqrt(mean((predictions_lasso - test_data$WinningPercentage)^2))
RMSE_values[5] = sqrt(mean((predictions_ridge - test_data$WinningPercentage)^2))

AIC_values[4:5] = NA
BIC_values[4:5] = NA

# METRICS INTO DATAFRAME
model_metrics = data.frame(
  Model = model_names,
  RMSE = RMSE_values,
  AIC = AIC_values,
  BIC = BIC_values
)

# DISPLAY METRICS
#print(model_metrics)

# CHOOSING BEST MODEL - BASED ON RMSE, AIC, BIC - Lower the better
#best_model_index = which.min(model_metrics$RMSE)
#best_model = model_metrics[best_model_index, ]
#print(paste("Best Model:", best_model$Model))

# MODEL 2 - LINEAR REGRESSION WITH TYPE COMBINATIONS
# FITTING LINEAR REGRESSION MODEL WITH TYPE COMBINATIONS
model2 = lm(WinningPercentage ~ type_combo, data = pokemon_data)
summary_model2 = summary(model2)

# PRINTING DESIRED STATISTICS
cat("Multiple R-squared: ", summary_model2$r.squared, "\n")
cat("Adjusted R-squared: ", summary_model2$adj.r.squared, "\n")
cat("F-statistic: ", summary_model2$fstatistic[1], " on ",
    summary_model2$fstatistic[2], " and ", summary_model2$fstatistic[3], " DF", "\n")
cat("p-value: ", pf(summary_model2$fstatistic[1], summary_model2$fstatistic[2],
    summary_model2$fstatistic[3], lower.tail = FALSE), "\n")

# BAR-PLOT FOR WINNING PERCETAGE BY TYPE COMBINATION
```

```r
par(mar = c(12, 4, 4, 2) + 0.1)
barplot(tapply(pokemon_data$WinningPercentage, pokemon_data$type_combo, mean),
        las = 2, cex.names = 0.7,
        xlab = "",
        ylab = "Winning Percentage",
        main = "Winning Percentage (by Type Combination)")
mtext("Type Combination", side = 1, line = 5)

# MODEL 3 - LINEAR REGRESSION WITH INTERACTION EFFECTS
model3 = lm(WinningPercentage ~ HP * Sp..Atk + Attack * Sp..Def + Defense * Speed,
            data = pokemon_data)
summary(model3)

#SCATTER PLOT WITH REGRESSION LINE FOR SELECTED INTERACTION EFFECT
par(mfrow = c(1, 3))

plot(pokemon_data$HP, pokemon_data$Sp..Atk,
     xlab = "HP", ylab = "Sp. Atk",
     pch = 20, col = "yellow", cex = 1.5)
abline(lm(pokemon_data$Sp..Atk ~ pokemon_data$HP), col = "darkred")

plot(pokemon_data$Attack, pokemon_data$Sp..Def,
     xlab = "Attack", ylab = "Sp. Def",
     pch = 20, col = "yellow", cex = 1.5)
abline(lm(pokemon_data$Sp..Def ~ pokemon_data$Attack), col = "darkred")

plot(pokemon_data$Defense, pokemon_data$Speed,
     xlab = "Defense", ylab = "Speed",
     pch = 20, col = "yellow", cex = 1.5)
abline(lm(pokemon_data$Speed ~ pokemon_data$Defense), col = "darkred")

# LINEARITY CHECK
plot(fitted(model3), resid(model3),
     main = "Residuals vs Fitted Values",
     xlab = "Fitted Values",
     ylab = "Residuals",
     pch = 20, col = "orange")
abline(h = 0, col = "blue", lw = 2)

# MODEL 4 - REGULARIZED REGRESSION
# LASSO REGRESSION
set.seed(42)
lasso_model = cv.glmnet(X, y, alpha = 1)
lasso_coef = coef(lasso_model, s = "lambda.min")
print(lasso_coef)
# PLOT LASSO COEFFICIENTS
par(mar = c(5.1, 4.1, 6.1, 2.1))
plot(lasso_model, xvar = "lambda", label = TRUE)
title("Lasso Regression Coefficients")
# MIN LAMBDA
lasso_lambda_min = lasso_model$lambda.min
print(lasso_lambda_min)
```

```r
lasso_grid1 = exp(seq(-1, 0, length = 100))
lasso_model1 = cv.glmnet(X, y, lambda = lasso_grid1)

plot(lasso_model1)

lasso_lambda_min1 = lasso_model1$lambda.min
print(lasso_lambda_min1)

# RIDGE REGRESSION
set.seed(42)
ridge_model = cv.glmnet(X, y, alpha = 0)
ridge_coef = coef(ridge_model, s = "lambda.min")
print(ridge_coef)
# PLOT RIDGE COEFFICIENTS
par(mar = c(5.1, 4.1, 6.1, 2.1))
plot(ridge_model, xvar = "lambda", label = TRUE)
title("Ridge Regression Coefficients")

# MINIMUM LAMBDA
ridge_lambda_min = ridge_model$lambda.min
print(ridge_lambda_min)

set.seed(42)

ridge_grid = exp(seq(log(ridge_lambda_min) - 1, log(ridge_lambda_min) + 1, length = 100))
ridge_model1 = cv.glmnet(X, y, alpha = 0, lambda = ridge_grid)

plot(ridge_model1)

ridge_lambda_min1 = ridge_model1$lambda.min
print(ridge_lambda_min1)
```